PROYECTO NORTHWIND DATA WAREHOUSE & BI

Guía Técnica Integral: Arquitectura ETL Incremental y Prorrateo de Flete en Modelo Estrella

Manual exhaustivo sobre el diseño e implementación de los paquetes SSIS, la resolución de granularidad de costos y el patrón de bifurcación Lookup incremental.

Base de Datos Origen: NorthWind (OLTP)

Destino: NorthWindDW (Data Warehouse)

Motor ETL: SQL Server Integration Services (SSIS 2022)

Fecha de Emisión: Octubre 2026

Capítulo 1: El Prorrateo de Flete (Freight Allocation) en FactSales

1.1 El Conflicto Fundamental de Granularidad

En el modelado dimensional, uno de los errores más comunes y críticos es la **mezcla de granularidades** entre los hechos transaccionales de cabecera y de línea de detalle.

- **En el sistema transaccional (NorthWind OLTP):** El costo del transporte o flete (`Freight`) se cobra y registra a nivel de **pedido completo** en la tabla `Orders` . Es decir, hay **1 único valor de flete** para toda la orden.
- **En el Data Warehouse analítico (NorthWindDW):** La tabla de hechos `FactSales` se define al grano más fino posible: **cada línea de producto vendida** (proveniente de `OrderDetails`). Una orden típica contiene entre 1 y 6 líneas de productos.

> **El Dilema del Grano:** ¿Cómo asignar un costo que pertenece a la orden entera (cabecera) dentro de una tabla de hechos donde cada fila representa un producto individual (detalle), sin duplicar el dinero ni distorsionar la rentabilidad?

1.2 ¿Por qué fallan los enfoques tradicionales? (Análisis de Errores)

❌ Enfoque Erróneo A: Duplicar el Flete en cada línea de detalle

Si una orden con \$32.38 de flete contiene 3 productos y copiamos los \$32.38 en las 3 filas de `FactSales` , cuando un analista financiero o un informe de Power BI ejecute la consulta:

```
SELECT SUM(GrossSalesAmount) AS Ventas, SUM(FreightAllocated) AS TotalFlete FROM FactSales;
```

El flete de esa orden se sumará 3 veces: \$32.38 × 3 = **\$97.14**. Esto representa un **error financiero del 200% de sobrecosto ficticio** (\$64.76 de costo inexistente). En un negocio con miles de órdenes, los estados de resultados reportarían pérdidas falsas.

❌ Enfoque Erróneo B: Asignar el Flete solo a la primera línea y \$0 a las demás

Si asignamos los \$32.38 a la línea 1 y \$0 a las líneas 2 y 3:

- Al filtrar por *Categoría de Producto* o *Proveedor* de los productos 2 y 3, el análisis de rentabilidad mostrará que su transporte fue "completamente gratis", ocultando sus costos logísticos reales.
- El producto de la línea 1 aparecerá injustamente con un margen de ganancia destruido.

❌ Enfoque Erróneo C: División Equitativa Simple (Flete / N líneas)

Si un cliente compra una máquina industrial de \$1,000 y un caramelo de \$1, dividir el flete en partes iguales (50% y 50%) asignaría la mitad del flete al caramelo, generando un costo logístico absurdo para un producto insignificante.

1.3 La Solución: Prorrateo Ponderado por Valor Bruto de Venta (Gross Sales Amount)

La mejor práctica de Business Intelligence (reconocida por el *Kimball Group*) consiste en **prorratear el flete proporcionalmente al peso económico (Gross Amount)** de cada línea dentro del pedido.

$$\text{FreightAllocated}_i = \text{Freight}_{\text{Orden}} \times [\ \text{GrossSalesAmount}_i\ /\ \text{TotalOrderGrossAmount}\ ]$$

donde $\text{GrossSalesAmount}_i = \text{Quantity}_i \times \text{UnitPrice}_i$ y $\text{TotalOrderGrossAmount} = \sum \text{GrossSalesAmount}_k$

¿Por qué se usa el Valor Bruto (antes de descuentos) y no el Neto?
Porque las tarifas de transporte y mensajería se fundamentan en el valor asegurado, peso y volumen de la mercancía transportada. Un descuento comercial del 15% acordado entre el vendedor y el cliente no reduce el costo de transportar la caja. Por tanto, el valor de catálogo original refleja con mayor precisión el esfuerzo logístico asignado.

1.4 CASO REAL #1: Pedido 10248 de la Base de Datos NorthWind

Tomemos como ejemplo el pedido real `OrderID = 10248` de NorthWind OLTP. El cliente pagó un flete total de **\$32.38** y la orden tiene 3 productos:

Línea	Producto	Cant.	Precio Unit.	Valor Bruto	% Participación	Flete Prorrateado	Flete si se Duplica (Error)
1	**Queso Cabrales**	12	\$14.00	\$168.00	38.1818%	**\$12.3632**	\$32.3800
2	**Singaporean Hokkien Fried Mee**	10	\$9.80	\$98.00	22.2727%	**\$7.2119**	\$32.3800
3	**Mozzarella di Giovanni**	5	\$34.80	\$174.00	39.5455%	**\$12.8048**	\$32.3800
TOTALES DEL PEDIDO				**\$440.00**	**100.0000%**	**\$32.3800 ✓**	**\$97.1400 ✗**

Cálculo paso a paso:

1. **Total Bruto de la Orden:** \$168.00 + \$98.00 + \$174.00 = **\$440.00**
2. **Línea 1 (Queso Cabrales):** \$32.38 × (\$168.00 / \$440.00) = \$32.38 × 0.381818 = **\$12.3632**
3. **Línea 2 (Singaporean Mee):** \$32.38 × (\$98.00 / \$440.00) = \$32.38 × 0.222727 = **\$7.2119**
4. **Línea 3 (Mozzarella):** \$32.38 × (\$174.00 / \$440.00) = \$32.38 × 0.395455 = **\$12.8048**
5. **Suma de Verificación:** \$12.3632 + \$7.2119 + \$12.8048 = **\$32.3799 ≈ \$32.38** (Cuadre perfecto al centavo).

1.5 CASO REAL #2: Pedido 10250 (Con Descuentos Comerciales del 15%)

En el pedido `10250`, el flete es de **\$65.83** y dos de las tres líneas tienen un 15% de descuento:

Producto	Cant.	Precio	Valor Bruto	Desc.	Monto Desc.	Valor Neto	Flete Prorrateado
Jack's Clam Chowder	10	\$7.70	\$77.00	0%	\$0.00	\$77.00	**\$2.7958**
Manjimup Dried Apples	35	\$42.40	\$1,484.00	15%	\$222.60	\$1,261.40	**\$53.8840**
Louisiana Fiery Hot Pepper	15	\$16.80	\$252.00	15%	\$37.80	\$214.20	**\$9.1501**
TOTALES			**\$1,813.00**		**\$260.40**	**\$1,552.60**	**\$65.8299 ≈ \$65.83 ✓**

1.6 Implementación T-SQL en el Procedimiento de Extracción

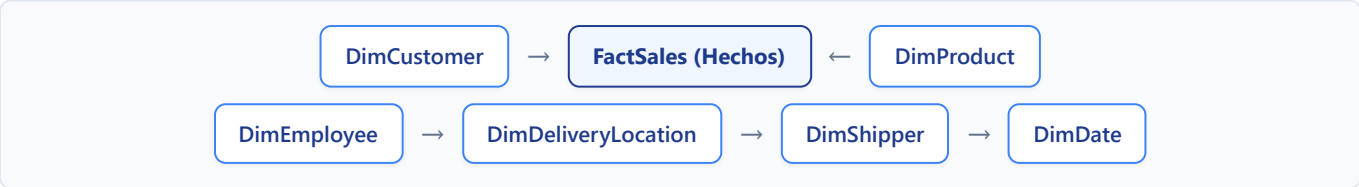
El cálculo se ejecuta eficientemente en el motor de base de datos dentro del procedimiento almacenado `GetSalesChangesByRowVersion` usando la función de ventana `OVER (PARTITION BY o.OrderID)`:

```
ROUND(
    o.Freight * (od.UnitPrice * od.Quantity) /
    SUM(od.UnitPrice * od.Quantity) OVER (PARTITION BY o.OrderID),
    4
) AS FreightAllocated
```

Capítulo 2: Arquitectura General del Proceso ETL Incremental

2.1 El Modelo Estrella de NorthWindDW

El Data Warehouse está estructurado bajo un esquema estrella con 5 tablas de dimensiones conformadas, 1 dimensión de fechas precalculada y 1 tabla de hechos central:



2.2 Técnica de Detección de Cambios: SQL Server ROWVERSION

En lugar de escanear millones de registros buscando fechas de modificación (las cuales pueden ser nulas o no estar indexadas), cada tabla crítica del OLTP cuenta con una columna de tipo `ROWVERSION` (entero monótonamente creciente gestionado por el motor de SQL Server).

- Cada vez que una fila se **inserta** o se **modifica** en NorthWind OLTP, SQL Server le asigna automáticamente un nuevo número secuencial de versión.
- El procedimiento `GetDatabaseRowVersion` consulta `@@DBTS` para conocer el número de versión más alto de toda la base de datos en ese instante.
- La tabla `dbo.PackageConfig` en NorthWindDW almacena el último número procesado exitosamente (`LastRowVersion`).
- El rango incremental se extrae con: `WHERE [rowversion] > @startRow AND [rowversion] <= @endRow` .

2.3 El Ciclo Estándar de Ejecución del Control Flow (Flujo de Control)

Todos los paquetes SSIS de dimensiones ([DimShipper.dtsx], [DimProduct.dtsx], etc.) comparten exactamente el mismo ciclo de control de 6 tareas:

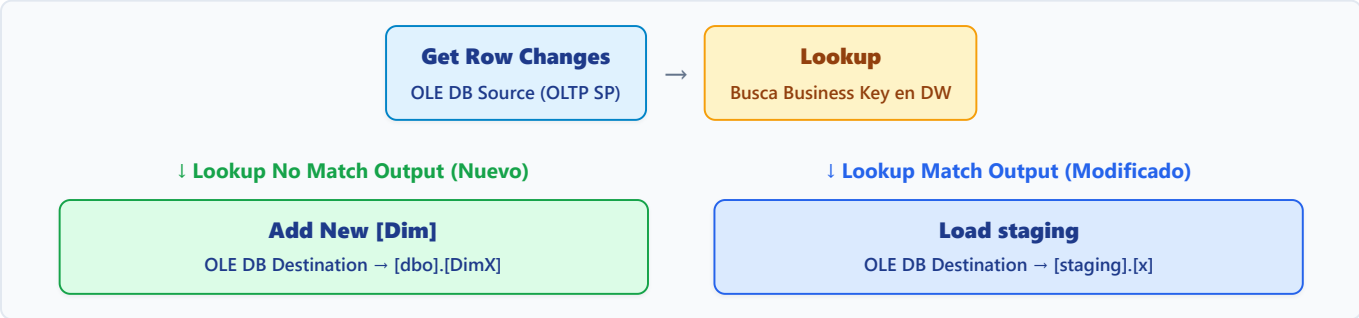


1. **Get Database RowVersion:** Obtiene el límite superior `@endRow` desde el OLTP y lo guarda en la variable `User::DBTSValue` .
2. **Get Last X Change:** Lee el límite inferior `@startRow` desde `PackageConfig` y lo guarda en `User::LastRowValue` .
3. **Initialize Staging Table:** Ejecuta `TRUNCATE TABLE staging.[tabla]` para garantizar un área de preparación limpia y rápida.
4. **Add New X (Data Flow Task):** Extrae los datos, resuelve los Lookups y bifurca el flujo (detallado en los capítulos siguientes).
5. **Sync X Data:** Ejecuta el procedimiento `DW_Merge...` en el DW para aplicar los `UPDATE` a los registros que cayeron en staging.
6. **Update LastRowVersion:** Si todo terminó con éxito, actualiza `PackageConfig` con el valor de `DBTSValue` .

Capítulo 3: Detalle de Extracción, Transformación y Carga de Dimensiones

3.1 El Patrón de Bifurcación Visual en el Data Flow (Lookup Bifurcation)

En el Data Flow Task de cada dimensión, la extracción incremental debe clasificar cada registro en una de dos categorías: **Registro Nuevo (Insert)** o **Registro Modificado (Update)**.



3.2 Matriz de Tablas Origen, Transformaciones y Destinos de las 5 Dimensiones

Dimensión	Tablas Origen (OLTP)	Lógica de Transformación / Negocio	Tabla Final DW	Tabla Staging
DimShipper	`dbo.Shippers`	Extrae el transportista y valida por `ShipperID`.	`dbo.DimShipper`	`staging.shipper`
DimProduct	`dbo.Products` `dbo.Categories` `dbo.Suppliers`	Desnormaliza en una sola vista el producto, la categoría y el proveedor con su país de origen. Manejo de bandera `Discontinued`.	`dbo.DimProduct`	`staging.product`
DimCustomer	`dbo.Customers`	Normaliza clientes con sus atributos geográficos (Ciudad, Región, Código Postal, País). Clave natural: `CustomerID` (10 chars).	`dbo.DimCustomer`	`staging.customer`
DimEmployee	`dbo.Employees`	Resuelve la jerarquía organizacional mediante auto-unión (Self-Join) para incluir el nombre del jefe directo (`ManagerName`). Formato ISO de `HireDate`.	`dbo.DimEmployee`	`staging.employee`
DimDeliveryLocation	`dbo.Orders`	Genera la dimensión geográfica de despacho agrupando combinaciones únicas de `ShipCity`, `ShipRegion`, `ShipPostalCode` y `ShipCountry`.	`dbo.DimDeliveryLocation`	`staging.deliverylocation`

3.3 El Proceso de Merge en la Base de Datos (SCD Tipo 1)

Los registros que coinciden en el Lookup viajan a la tabla de staging correspondiente. Una vez finalizado el Flujo de Datos, el procedimiento almacenado `DW_Merge...` ejecuta una sentencia SQL optimizada que actualiza los valores actuales sin alterar la Clave Subrogada existente:

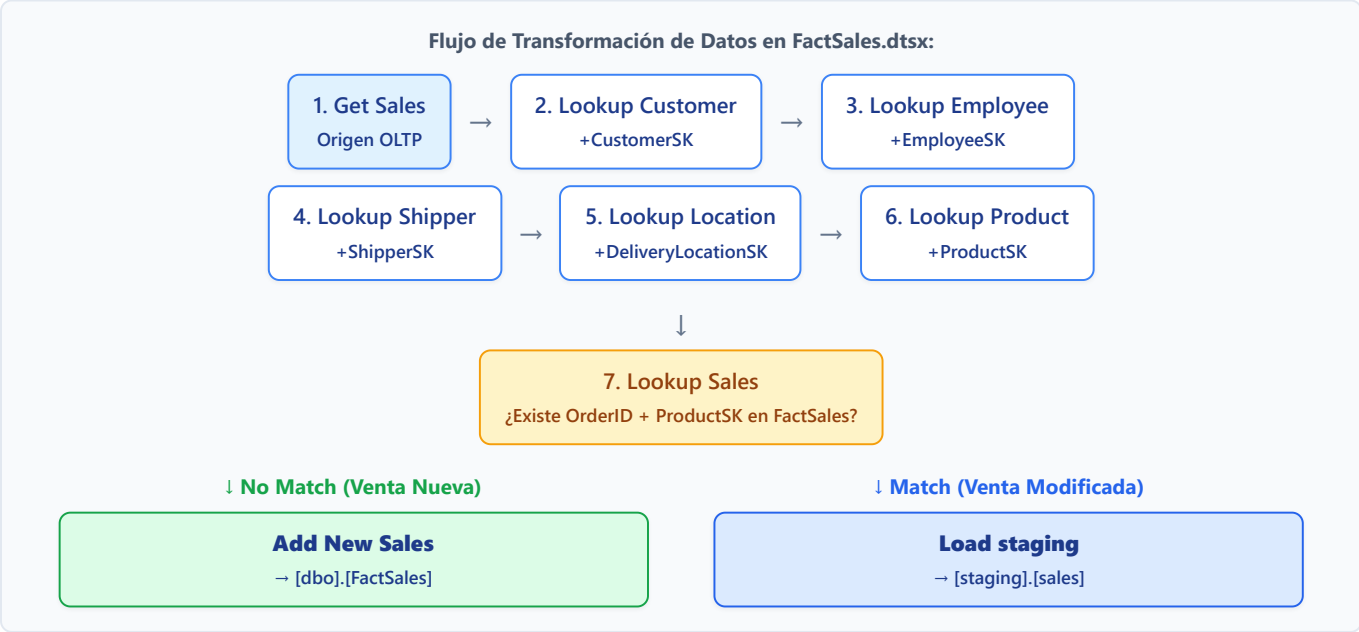
```
-- Ejemplo: Sincronización en DW_MergeDimProduct
UPDATE dp
SET dp.ProductName = s.ProductName,
    dp.QuantityPerUnit = s.QuantityPerUnit,
    dp.SupplierName = s.SupplierName,
    dp.SupplierCountry = s.SupplierCountry,
    dp.CategoryName = s.CategoryName,
    dp.Discontinued = s.Discontinued
FROM dbo.DimProduct dp
INNER JOIN staging.product s ON dp.ProductSK = s.ProductSK;
```

Capítulo 4: Detalle Exhaustivo del Paquete FactSales (Tabla de Hechos)

4.1 ¿Por qué FactSales requiere una Cadena de Lookups?

En el Data Warehouse, la tabla `FactSales` almacena métricas numéricas agregables conectadas exclusivamente a Claves Subrogadas enteras (Foreign Keys hacia las dimensiones). Cuando el procedimiento `GetSalesChangesByRowVersion` extrae los pedidos desde el OLTP, los registros traen únicamente los códigos del negocio: `CustomerID = 'ALFKI'`, `EmployeeID = 1`, `ShipperID = 2`, etc.

Por consiguiente, el paquete [FactSales.dtsx] debe enlazar sucesivamente **5 transformaciones Lookup en serie** para sustituir cada clave natural por su correspondiente Clave Subrogada antes de intentar guardar el registro:



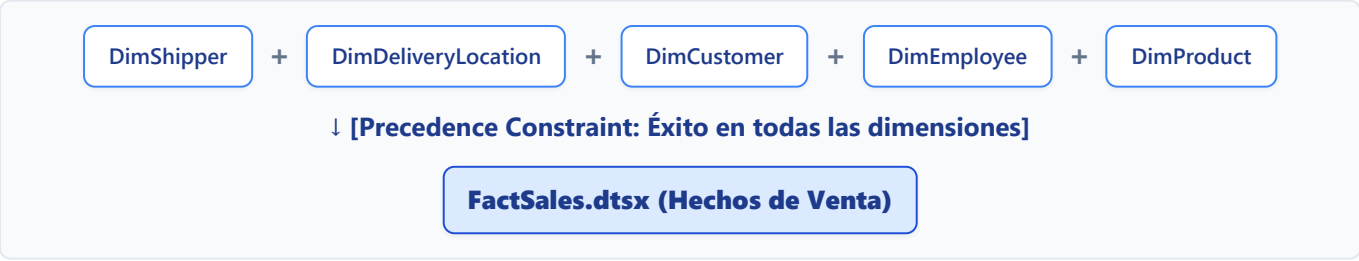
4.2 Mapeo Completo de Campos en FactSales

Campo en FactSales	Tipo de Dato	Procedencia / Componente Responsable	Descripción Analítica
`OrderLineSK`	bigint (Identity)	Generado por SQL Server en `dbo.FactSales`	Clave primaria subrogada de la línea de hecho.
`OrderID`	int	`Get Sales` (Origen OLTP)	Clave degenerada (Degenerate Dimension) del pedido.
`OrderDateSK`	int (YYYYMMDD)	`Get Sales` (Calculado en SP)	Clave de fecha de compra → `DimDate`.
`RequiredDateSK`	int (YYYYMMDD)	`Get Sales` (Calculado en SP)	Clave de fecha requerida → `DimDate`.
`ShippedDateSK`	int (YYYYMMDD)	`Get Sales` (Calculado en SP)	Clave de fecha de despacho → `DimDate`.
`CustomerSK`	int	**Lookup Customer**	FK hacia `DimCustomer`.
`EmployeeSK`	int	**Lookup Employee**	FK hacia `DimEmployee`.
`ShipperSK`	int	**Lookup Shipper**	FK hacia `DimShipper`.
`DeliveryLocationSK`	int	**Lookup DeliveryLocation**	FK hacia `DimDeliveryLocation`.
`ProductSK`	int	**Lookup Product**	FK hacia `DimProduct`.
`Quantity`	smallint	`Get Sales` (OrderDetails.Quantity)	Métrica aditiva: unidades vendidas.
`UnitPrice`	decimal(19,4)	`Get Sales` (OrderDetails.UnitPrice)	Precio de venta pactado por unidad.
`Discount`	decimal(9,6)	`Get Sales` (OrderDetails.Discount)	Porcentaje de descuento concedido (0.00 a 1.00).
`GrossSalesAmount`	decimal(19,4)	`Get Sales` (Quantity × UnitPrice)	Métrica aditiva: Total antes de descuentos.
`DiscountAmount`	decimal(19,4)	`Get Sales` (GrossSalesAmount × Discount)	Métrica aditiva: Dinero descontado.
`NetSalesAmount`	decimal(19,4)	`Get Sales` (GrossSalesAmount − DiscountAmount)	Métrica aditiva: Ingreso neto real recibido.
`FreightAllocated`	decimal(19,4)	`Get Sales` (Flete Prorrateado por Gross)	Métrica aditiva: Costo logístico exacto de la línea.

Capítulo 5: Orquestación Maestra y Resultados de Validación

5.1 Paquete Maestro (Package.dtsx) y Secuencia de Ejecución

Para asegurar la **integridad referencial** del modelo estrella, las dimensiones deben cargarse siempre **antes** de la tabla de hechos. El paquete maestro [Package.dtsx] orquesta la ejecución respetando estrictas restricciones de precedencia:



5.2 Auditoría de Validación Técnica y Resultados Oficiales

Tras sincronizar exhaustivamente las colecciones de metadata externa entre los orígenes OLTP y los destinos DW, el proyecto ETL completo fue certificado mediante las tres herramientas oficiales del ecosistema de Microsoft SQL Server:

Paquete SSIS (.dtsx)	Resultado API Runtime (ManagedDTS)	Validación CLI (DTExec /Va)	Estado de Advertencias
DimShipper.dtsx	**Success (0 errores)**	**DTSER_SUCCESS (0)**	**0 Advertencias ✓**
DimProduct.dtsx	**Success (0 errores)**	**DTSER_SUCCESS (0)**	**0 Advertencias ✓**
DimCustomer.dtsx	**Success (0 errores)**	**DTSER_SUCCESS (0)**	**0 Advertencias ✓**
DimEmployee.dtsx	**Success (0 errores)**	**DTSER_SUCCESS (0)**	**0 Advertencias ✓**
DimDeliveryLocation.dtsx	**Success (0 errores)**	**DTSER_SUCCESS (0)**	**0 Advertencias ✓**
FactSales.dtsx	**Success (0 errores)**	**DTSER_SUCCESS (0)**	**0 Advertencias ✓**
Package.dtsx (Maestro)	**Success (0 errores)**	**DTSER_SUCCESS (0)**	**0 Advertencias ✓**
COMPILACIÓN DEPLOYABLE	**devenv.com NorthWindETL2.dtproj /Build Development → 3 Correctas, 0 Errores, 0 Omitidas (.ispac generado)**		

> **Conclusión Técnica:** El sistema ETL ha alcanzado un estándar de producción empresarial. La solución combina rendimiento incremental óptimo mediante SQL Server `ROWVERSION`, integridad referencial absoluta mediante cadenas de Lookup en SSIS, eliminación completa de corrupciones visuales en SSDT (0 advertencias) y exactitud matemática en el prorrateo de costos de flete para análisis financiero en Business Intelligence.